

Metodologie di Programmazione (A-L)

II semestre - a.a. 2025 – 2026

Stream II

a cura di Massimo La Morgia



SAPIENZA
UNIVERSITÀ DI ROMA

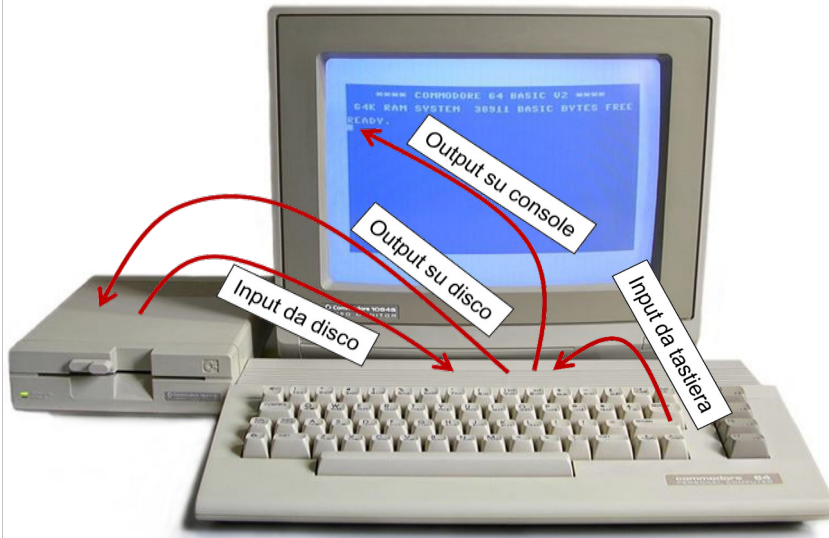
*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

**I crediti sulle slide di questo corso sono riportati nell'ultima slide

Stream<T> parte II

Input & Output

Stream<T>



Gli **Stream** di I/O sono un concetto ben distinto da **Stream API**.

Metodi di `java.util.stream.Stream`: `min` e `max` (terminali)

- I metodi `min` e `max` restituiscono rispettivamente il minimo e il massimo di uno stream sotto forma di `Optional`
- Prendono in input un `Comparator` sul tipo degli elementi dello stream

```
List<Integer> p = Arrays.asList(2, 3, 4, 5, 6, 7);  
Optional<Integer> max = p.stream().max(Integer::compare);  
  
// se c'è, restituisce il massimo; altrimenti restituisce -1  
System.out.println(max.orElse(-1));
```

Metodi di `java.util.stream.Stream`: `filter` (intermedia), `forEach` (terminale)

- `filter` è un metodo di `Stream` che accetta un predicato (`Predicate`) per filtrare gli elementi dello stream
 - Operazione intermedia che restituisce lo stream con i soli elementi che rendono true il predicato specificato.
- `forEach` prende in input un `Consumer` e lo applica a ogni elemento dello stream
 - Operazione terminale, e dal momento che ha il ruolo di consumare gli elementi dello stream non restituirà alcun oggetto.



Esempi di Stream (con filter e forEach)

- Filtra gli elementi di una lista per iniziale e stampa ciascun elemento rimanente:

```
List<String> l = Arrays.asList("da", "ab", "ac", "bb");  
l.stream() // Stream<String>  
.filter(s -> s.startsWith("a")) // Stream<String>  
.forEach(System.out::println); // non abbiamo un tipo atteso
```

- Filtra gli elementi di una lista di interi mantenendo solo quelli dispari e stampa ciascun elemento rimanente:

```
List<Integer> lInt = Arrays.asList(4, 8, 15, 16, 23, 42);  
lInt.stream() // Stream<Integer>  
.filter(k -> k % 2 == 1) // Stream<Integer>  
.forEach(System.out::println); // non abbiamo un tipo atteso
```



Altro esempio di Stream (con filter e forEach)

- Filtra gli elementi di una lista per iniziale e lunghezza della stringa e stampa ciascun elemento rimanente:

```
Predicate<String> startsWithJ = s -> s.startsWith("J");  
Predicate<String> fourLetterLong = s -> s.length() == 4;
```

```
List<String> l3 = Arrays.asList("Java", "Scala", "Lisp");  
l3.stream() // Stream<String>  
  .filter(startsWithJ.and(fourLetterLong)) // Stream<String>  
  .forEach(s -> System.out.println("Inizia con J ed e' lunga 4  
caratteri: "+s)); // non abbiamo un tipo atteso
```

Metodi di `java.util.stream.Stream`: `count` (terminale)

- `count` è un'operazione terminale che restituisce il numero `long` di elementi nello stream
- Esempio: (sia `l` il riferimento ad una collezione di `String`)

```
long startsWithA = l.stream() // Stream<String>
```

```
    .filter(s -> s.startsWith("a")) // Stream<String>
```

```
    .count(); // tipo atteso long
```

```
System.out.println(startsWithA);
```

- Esempio di conteggio del numero di righe di un file di testo:

```
long numberOfLines = Files.lines(Paths.get("yourFile.txt")) // Stream<String>
```

```
    .count(); // tipo atteso long
```


Metodi di java.util.stream.Stream: sorted (intermedia)

- **sorted** è un'operazione intermedia sugli stream che restituisce una **vista ordinata** dello stream **senza modificare la collezione sottostante**
- Esempio:

```
List<String> l = Arrays.asList("da", "ac", "ab", "bb");
```

```
l.stream() // Stream<String>
```

```
.sorted() // Stream<String>
```

```
.filter(s -> s.startsWith("a")) // Stream<String>
```

```
.forEach(System.out::println); // nessun tipo atteso
```

- Stampa:

```
ab
```

```
ac
```

Metodi di `java.util.stream.Stream`: `map` (intermedia)

- `map` è un'operazione intermedia sugli stream che restituisce un nuovo stream in cui **ciascun elemento dello stream di origine è convertito in un altro oggetto** attraverso la funzione (`Function`) passata in input
- Esempio: restituire tutte le stringhe, in maiuscolo e ordinate in ordine inverso.

```
l.stream() // Stream<String>
```

```
.map(String::toUpperCase) // Stream<String> equivalente a .map(s -> s.toUpperCase())
```

```
.sorted(Comparator<String>.naturalOrder().reversed()) // Stream<String>
```

```
.forEach(System.out::println);
```

Stampa:

DA

BB

AC

AB



java.util.stream.Stream.map: esempio

- Si vuole scrivere un metodo che aggiunga l'IVA (22%) a ciascun prezzo:

```
List<Integer> ivaEsclusa = Arrays.asList(10, 20, 30);
```

// In Java 7:

```
for (int p : ivaEsclusa)
{
    double plvaInclusa = p*1.22;
    System.out.println(plvaInclusa);
}
```

// In Java 8:

```
ivaEsclusa.stream() // Stream<Integer>
               .map(p -> p*1.22) // Stream<Double>
               .forEach(System.out::println); // non abbiamo un tipo atteso
```

Metodi di `java.util.stream.Stream`: `collect` (terminale)

- `collect` è un'operazione terminale che permette di raccogliere gli elementi dello stream in un qualche oggetto (ad es. una `collection`, una `stringa`, un `intero`)
- Ad esempio, per ottenere la lista dei prezzi ivati:

```
List<Integer> ivaEsclusa = Arrays.asList(10, 20, 30);
```

// In Java 7:

```
List<Double> l = new ArrayList<>();  
for (int p : ivaEsclusa) l.add(p*1.22);
```

// In Java 8:

```
List<Double> l = ivaEsclusa.stream() // Stream<Integer>  
    .map(p -> p*1.22) // Stream<Double>  
    .collect(Collectors.toList()); // List<Double>
```



Esempio: trasformare una lista di stringhe in una lista delle lunghezze delle stesse

```
List<String> words = Arrays.asList("Oracle", "Java",  
"Magazine");
```

```
List<Integer> wordLengths =  
    words.stream() // Stream<String>  
        .map(String::length) // Stream<Integer>  
        .collect(Collectors.toList()); // List<Integer>
```

java.util.stream.Collectors

- "Ricette" per ridurre gli elementi di uno stream e raccogliarli in qualche modo
- Per rendere più leggibile il codice:
`import static java.util.stream.Collectors.*`
 - In questo modo possiamo scrivere il nome del metodo senza anteporre Collectors. (es. `toList()` invece di `Collectors.toList()`)

java.util.stream.Collectors: riduzioni a singolo elemento

- `counting()` – restituisce il numero di elementi nello stream (risultato di tipo `long`)

```
List<Integer> l = Arrays.asList(2, 3, 5, 6);
```

```
// k == 2
```

```
long k = l.stream().filter(x -> x < 5).collect(Collectors.counting());
```

- `maxBy/minBy(comparator)` – restituisce un `Optional` con il massimo/minimo valore

```
// max contiene 6
```

```
Optional<Integer> max = l.stream().collect(maxBy(Integer::compareTo));
```

- `summingInt`(lambda che mappa ogni elemento a intero)/`averagingInt`, `summingDouble`, `averagingDouble`

java.util.stream.Collectors: riduzioni a singolo elemento

- `joining()`, `joining(separatore)`, `joining(separatore, prefisso, suffisso)` – concatena gli elementi stringa dello stream in un'unica stringa finale

```
List<Integer> l = Arrays.asList(2, 3, 5, 6, 2, 7);
```

```
String str = l.stream().map(x -> ""+x).collect(joining(","));
```

```
// str.equals("2,3,5,6,2,7")
```

- `toList`, `toSet` e `toMap` – accumulano gli elementi in una lista, insieme o mappa (non c'è garanzia sul tipo di List, Set o Map)

```
Set<String> set = l.stream().map(x -> ""+x).collect(toSet());
```

- `toCollection` – accumula gli elementi in una collezione scelta

```
ArrayList<String> str = l.stream().map(x -> ""+x)  
                           .collect(toCollection(ArrayList::new));
```


Collectors.toMap: riduzione a una mappa

- **toMap** prende in input fino a 4 argomenti:
 - la **funzione** (Function) per mappare l'oggetto dello stream nella chiave della mappa
 - la **funzione** (Function) per mappare l'oggetto dello stream nel valore della mappa
 - **opzionale**: la **Biunzione** (BiFunction) da utilizzare per unire il valore preesistente nella mappa a fronte della chiave con il valore associato all'oggetto dalla seconda funzione (non devono trovarsi due chiavi uguali o si ottiene un'eccezione **IllegalStateException**)
 - **opzionale**: il **Supplier** che fornisce il riferimento ad un costruttore e serve a specificare quale tipo di mappa

```
Map<Integer, String> map = persons
```

```
.stream() // Stream<Person>
```

```
.collect(Collectors.toMap(
```

```
    Person::getAge, // ottiene la chiave
```

```
    Person::getName, // ottiene il valore
```

```
    (name1, name2) -> name1 + ";" + name2)); // combina due valore
```

Credits

Le slide di questo corso sono basate sulle slide del Prof. Faralli che sono a loro volta una rielaborazione delle slide del Prof. Navigli

In aggiunta, le slide sono state revisionate dagli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica: Mario Marra e Paolo Straniero.

Font ad alta leggibilità Biancoenero® di biancoenero edizioni srl, disegnata da Umberto Mischì, con la consulenza di Alessandra Finzi, Daniele Zanoni e Luciano Perondi (Brevetto n. RM20110000128).

Disponibile gratuitamente per tutte le istituzioni e i privati che la utilizzino per scopi non commerciali.